

Dops Concepts



Object Oriented Programming

Without OOPs



Banking

Withdrawal

Deposit

account_balance



hard to maintain
codebase

With OOPs → easy to
maintain

Class Account:

Attribute

↳ account_number

balance

Function

↳ deposit

Withdraw

The Need for OOPs

↳ Reusability

↳ Manage Complexity

↳ Extensibility → if you want

to extend the features/
function

Properties of OOPs Programming

① Reusability → Inheritance

Base class (Parent class)



Derived class (child class)

Parent/
Base
class

Account

deposit ✓

Withdraw ✓

display ✓

Savings

current

→ Overdraft

Limit

Derived

class ←

② Polymorphism → same function

but different

behaviour based on

what specific class object it

belongs to

③ Encapsulation

Encapsulation in Python is a fundamental principle of Object-Oriented Programming (OOP) that involves bundling data (attributes) and methods that operate on that data into a single unit, a class. This mechanism is primarily used to control access to the internal state of an object, preventing accidental modification and hiding the implementation details from the outside world

class definition

class Account:

def __init__(self, account_number, balance):

self.account_number = account_number

self.balance = balance

Attribute

entered by the user

adding the money to the account

def deposit(self, amount):

self.balance += amount

withdraw the money from the account

def withdraw(self, amount):

if amount <= self.balance:

self.balance -= amount

else:

print("Insufficient funds")

display the account details

def display(self):

print(f"Account Number: {self.account_number}, Balance:

{self.balance}")

The super() function is used to call methods from a parent (superclass) inside a child (subclass)

Inheritance: SavingsAccount inherits from the Account

class SavingsAccount(Account):

def __init__(self, account_number, balance, interest_rate):

super().__init__(account_number, balance)

self.interest_rate = interest_rate

access the attribute

& functions of the

base class

new attribute

display the savings account details

polymorphism

def display(self):

print(f"Saving Account Number: {self.account_number}, Balance:

{self.balance}, Interest Rate: {self.interest_rate}")

add the interest in the savings account

def add_interest(self):

interest = self.balance * self.interest_rate / 100

self.balance += interest

print(f"Interest added: {interest}. New Balance: {self.balance}")

create the object of the class

acc_priya = SavingsAccount(account_number=101, balance=10000, interest_rate=3)

acc_harsh = Account(account_number=102, balance=30000)

acc_priya.deposit(2000)

acc_priya.add_interest()

acc_priya.display()

{ balance → 1,00,000
Overdraft Limit → 20,000

An overdraft limit is a pre-approved maximum amount a bank allows you to withdraw from your account, even if your balance is zero, essentially a short-term credit line for managing cash flow gaps, with interest charged only on the utilized amount, not the full limit.

④ Abstraction
↳ Decorators

Abstraction in Python is a core principle of object-oriented programming (OOP) that involves hiding unnecessary implementation details and exposing only the essential features or functionality to the user. The primary goal is to manage complexity, improve code modularity, and allow the user to focus on *what* an object does rather than *how* it does it. 

↳ balance → private
Encapsulation → how data is
protected

abstraction → what functionality
is exposed
↳ hiding implementation details
& expose what the
user needs